EPOCH AI / ML CLUB

IIIT SRI CITY

BUILDING YOUR FIRST ML PROJECT

From a real-world question → to a working prediction

RUNNING EXAMPLE · PREDICT A STUDENT'S FINAL EXAM MARKS

Problem → Data → Model → Evaluation → Deployment

So... where do we actually start?

Not with Linear Regression. Not with a fancy model. Start with the question.

THE REAL TASK

Turn a real-world question

into a prediction you can trust.

Example → “Can we predict a student’s final exam marks?”

TODAY’S ROUTE

01	**QUESTION**	What are we predicting?
02	**DATA**	What do we have?
03	**EVIDENCE**	What is messy / useful?
04	**MODEL**	What fits the task?
05	**EVALUATE**	Does it generalize?
06	**IMPROVE**	What should change next?

The model is one decision inside a much bigger reasoning process.

1 · Problem formulation

Before code: define the prediction.

“Can we predict a student's final exam marks?”

INPUT FEATURES X

hours_studied
hours_slept
attendance
previous_marks

MODEL

learns a mapping

$y = f(X)$

TARGET y

final_marks

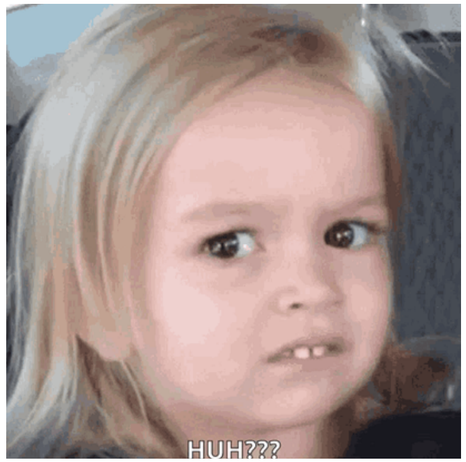
continuous number

REGRESSION because the output is a continuous numerical value.

2 · Collect data

Each row is one student's academic profile.

student_id	hours_studied	attendance	prev_marks	final_marks
101	2	70%	45	51
102	4	80%	55	63
103	6	90%	68	76
104	8	95%	78	88



DATASET ARRIVES: “NOW WHAT?”

WHY NOT student_id?

Keep it for tracking; exclude it from X because it does not provide predictive signal for marks.

ENGINEER'S QUESTION

Is the data relevant, reliable, and actually connected to the outcome?

3 · Understand the dataset

EDA is where surprises become visible.

SHAPE

rows / columns

TYPES

numeric? text? dates?

RANGES

are values plausible?

MISSING

where are NaNs?

DUPS

duplicates?

REAL DATA IS MESSY

"six" instead of 6

inspect → understand → fix

NaN values

inspect → understand → fix

850% attendance

inspect → understand → fix

Male / M / MALE

inspect → understand → fix

4 · Features + target

Decide what the model is allowed to see.

FEATURES X

hours_studied
hours_slept
attendance
prev_marks

These are the inputs available before prediction.

TARGET y

final_marks

This is the ground-truth outcome we want to predict.

Never leak the target into X.

X = INPUTS

y = OUTPUT

5 · Preprocessing is driven by evidence

Don't blindly apply a checklist.

OBSERVED PROBLEM	APPROPRIATE FIX
"six" → 6	convert wrong type
850% → valid range	fix / investigate
NaN	fill or otherwise handle
Male/M/MALE	normalize categories

The goal is clean, consistent input — not “more preprocessing.”

6 · Train / test split

Generalize. Don't memorize.

TRAINING DATA

Used to learn the model's parameters.

The model is allowed to see these rows during training.

TEST DATA

Held back until evaluation.

It represents unseen data and checks generalization.

TRAIN **TEST**

Tuning uses validation / cross-validation — don't optimize your final test score.

7 · Regression or classification?

The target determines the task type.

REGRESSION

Predict a continuous number.

Example:
Hours studied + attendance + previous marks
→ final marks = 78.5

Typical example model: Linear Regression

CLASSIFICATION

Predict a category / label.

Example:
CGPA + projects + internships + coding score
→ placement = YES / NO

Typical example model: Logistic Regression

8 · Choose a model family

Start simple, then compare alternatives.

Linear Regression

continuous values

Logistic Regression

classification

Decision Trees

rule-like splits

SVM

separation / margins

Neural Networks

complex patterns

Model choice is part of the reasoning — not a random algorithm lottery.

9 · Training → the model learns

Fit parameters by reducing prediction error.

X + y

training examples

TRAINING ALGORITHM

predict → measure error →
adjust parameters → repeat

FITTED MODEL

learned weights / coefficients

Each parameter controls how much a feature influences the prediction.

LOSS / ERROR

10 · Predict on unseen inputs

Now the model has to perform outside its training rows.

NEW STUDENT

hours_studied = 6
hours_slept = 7
attendance = 90%
prev_marks = 68

TRAINED MODEL

same preprocessing
+
learned parameters

PREDICTION

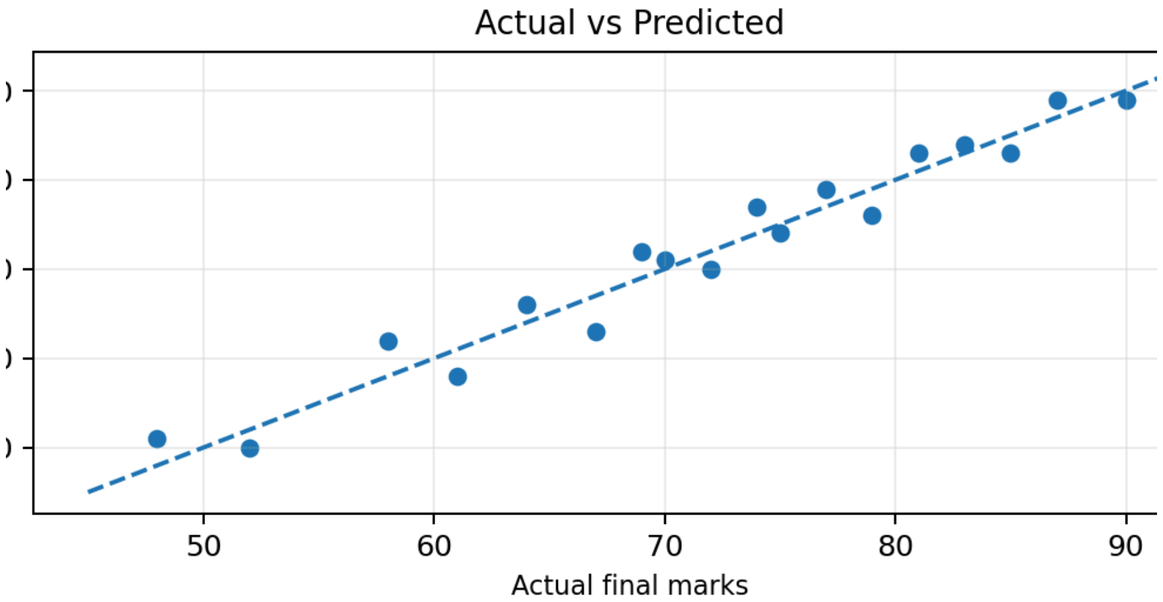
Predicted final marks

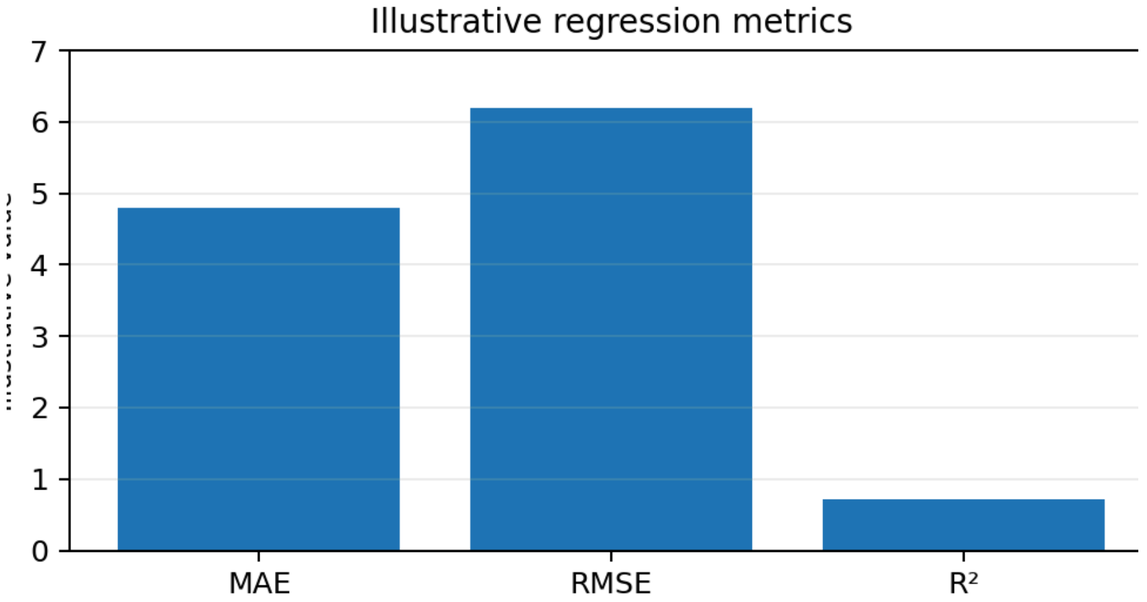
≈ 74

Then compare against actual marks.

11 · Evaluation: how wrong are we?

Different tasks need different metrics.





REGRESSION

MAE → average absolute error | RMSE → larger errors matter more | R^2 → explained variation

12 · Classification metrics

Accuracy is not the whole story.

Accuracy

overall correctness

Precision

when we predict positive, how often are we right?

Recall

how many actual positives did we catch?

F1

balances precision + recall

Choose the metric based on what kinds of errors matter.

Accuracy can fool you

A model can look “95% correct” while being useless for the thing you care about.

WHEN ACCURACY LOOKS GREAT...



“95% accuracy?!”

THE DUMB MODEL

Predict “NOT PLACED” for everyone.

100 students → only 5 are placed

95 correct • 5 missed

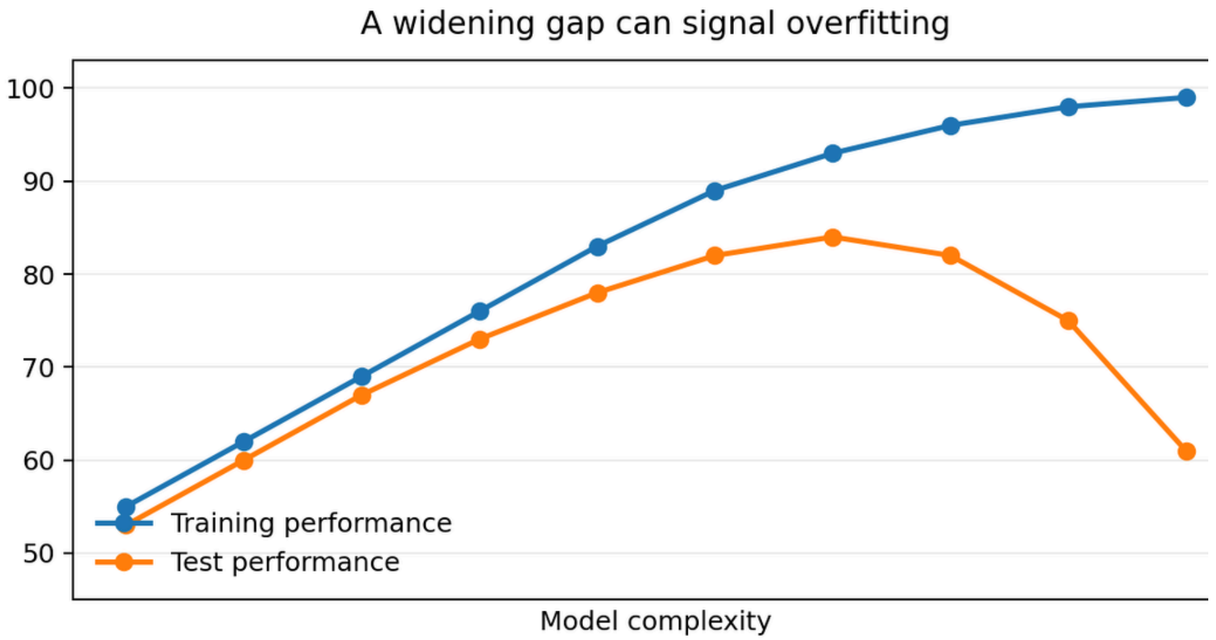
95% ACCURACY

0% RECALL

The model never finds a placed student. → Always check the metric against the real goal.

Error diagnosis

Compare train vs test performance.



UNDERFIT

Train 62%
Test 60%

Too simple

GOOD FIT

Train 88%
Test 85%

Small gap

OVERFIT

Train 99% → Test 58%

The model memorized training rows instead of learning a pattern.

THE GAP IS A DIAGNOSTIC SIGNAL

Overfitting

Train 99% → Test 58%: the model memorizes instead of generalizing.



TRAIN

99% — extremely comfortable with data it has already seen.

TEST

58% — unseen data exposes weak generalization.

MEMORIZE ≠ LEARN

The improvement loop

Change ONE thing, then check whether it actually helped.

TRAIN

Learn patterns from the training data.

EVALUATE

Measure performance using validation data and suitable metrics.

DIAGNOSE

Find what is going wrong: data issues, underfitting or overfitting.

CHANGE ONE THING

Try better features, preprocessing, a different model or tuning.

DATA QUALITY

clean issues, add useful signal

PREPROCESSING

scaling, encoding, missing data

MODEL / TUNING

family, hyperparameters, regularization

What can we change?

Improvement is a controlled experiment.

DATA

fix invalid values
handle missing values
remove/cap outliers

FEATURES

add useful features
drop irrelevant ones

MODEL FAMILY

try trees, SVM,
other families

TUNING

Grid Search
Random Search

REGULARIZATION

L1 / Lasso → sparsity
L2 / Ridge → shrink coefficients

RULE

Change ONE thing → retrain → reevaluate.
A higher train score alone is not proof.

Deployment

The same pipeline now serves new data.

NEW INPUT

new student row

PREPROCESS

same cleaning / encoding / scaling

MODEL

trained model

PREDICTION

final output

MONITOR

performance over time

DRIFT

input data changes

RETRAIN

update with new data

Final challenge · Will a student get placed?

Apply the same thinking to a new problem.

student_id	**CGPA**	**Attendance**	**Num_Projects**
Internships	**Coding_Score**	**Communication_Rating**	**Placement_Status**

YOUR QUESTIONS

1. What is the target?
2. Which columns are useful / excluded?
3. What data-quality issues would you inspect?
4. What preprocessing is needed?
5. Regression or classification?
6. Which model family first?
7. How would you split the data?
8. Which metrics?
9. Train 99%, Test 58% — what happened?
10. What would you change next?

THE TAKEAWAY

When you get a new ML problem...

Don’t ask “Which algorithm should I use?” first.

01 **What is the question?**

02 **What are X and y?**

03 **What does the data tell me?**

04 **How will I know the model is good?**

05 **What should I change next?**

Question → Data → Evidence → Decision → Model → Evaluation → Improvement

Questions? Let’s build one.